Will AI mess my RTL? A snapshot-based approach to verifying AI-written RTL

Bernat Homs[1,2]
bernat.homsgisbert@bsc.es

Oscar Palomar[2]
oscar.palomar@bsc.es

Miquel Moreto[1,2]
miquel.moreto@bsc.es

Marcelo Orenes-Vera[3]
morenesvera@nvidia.com

[1]Universitat Politècnica de Catalunya, Barcelona, Spain
[2]Barcelona Supercomputing Center, Barcelona, Spain
[3]NVIDIA, Santa Clara, USA

arXiv:2611.12712v1 [cs.SE] 9 Nov 2026

Abstract

With the growing interest in using AI (Artificial Intelligence) for RTL (Register-Transfer Level) hardware development, robust and comprehensive verification has become more important than ever. As Large Language Models (LLMs) increasingly assist in creating and modifying RTL designs, ensuring that these changes preserve existing functionality is paramount for building trust in AI-driven workflows. We present SVApshot, a fully automated framework that leverages LLMs to generate and systematically attempt to correct SystemVerilog Assertions (SVA) for RTL modules, completely removing the human from the generation loop. SVApshot introduces a novel snapshot methodology that captures the current functionality of a design as a comprehensive set of formal assertions, creating a regression suite for validating future changes—whether made by humans or AI. The framework features automated duplicate detection, assertion set expansion, and iterative repair of failing assertions using LLM-guided debugging with formal verification feedback. Our experimental evaluation across diverse RTL modules demonstrates high coverage for complex modules, with successful detection of both manually-injected and AI-introduced bugs. Additionally, this framework opens the door for automatic generation of formal testbenches to benchmark LLMs for hardware design, moving beyond binary pass/fail metrics towards nuanced assertion-based scoring.

1 Introduction and Background

The rapid advancement of LLMs has opened a new era of AI-driven RTL design, where automated tools increasingly handle the creation and modification of hardware descriptions [1, 11, 13], introducing unprecedented verification challenges. As LLMs automate complex design tasks, ensuring that AI modifications to RTL preserve existing functionality while introducing new features has become paramount for building trust in AI-driven workflows. Traditional approaches to automated assertion generation have made significant strides but fall short of addressing the unique demands of AI-assisted hardware design. Early methods like GoldMine [12] pioneered dynamic assertion mining through simulation-based data mining, while template-based approaches such as HARM [8] and A-TEAM [6] leveraged predefined patterns to generate assertions from execution traces. ILA [9] generates a model of the design from a functional specification and compares it against its RTL implementation, and AutoSVA [20] introduced annotation-driven

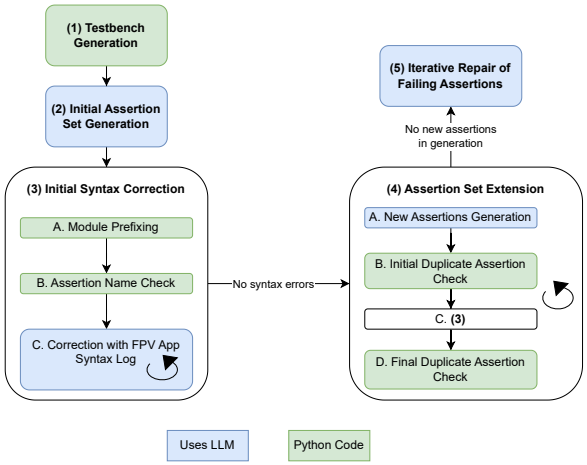

Figure 1: SVApshot toolflow diagram showing the complete process from RTL input to final snapshot generation. Green boxes represent automated processes scripted in Python, blue boxes represent processes involving LLMs.

formal testbench generation. However, these conventional methods are constrained by their dependence on simulation coverage, predefined templates, models, or manual annotations. The emergence of LLM-based assertion generation has shown promise in addressing some of these limitations [21, 23]. Recent works like AssertLLM [23] process complete specification documents to generate assertions, while AssertionForge [2] constructs knowledge graphs from both specifications and RTL code to discover complex signal interactions. Spec2Assertion [16] focuses on pre-RTL assertion generation from architectural specifications. Despite these advances, current LLM-based approaches suffer from critical gaps: they lack mechanisms for iterative refinement of failing assertions and do not provide systematic approaches for maintaining assertions across design evolution. Our tool, **SVApshot**, is designed to be used in the rapid, iterative nature of AI-driven design. It addresses these limitations by introducing a framework focused on ***creating a snapshot that captures the functionality of an RTL module as a comprehensive set of SystemVerilog Assertions (SVA) that serves as a regression suite for future modifications.*** Unlike existing approaches that generate assertions from specifications, SVApshot leverages the goal of treating the current RTL as a golden reference to iteratively refine LLM-generated assertions and obtain a set of proven assertions with respect to the reference RTL. Specifications are often incomplete and expressed in diverse, non-textual forms such as waveforms or tables, which standard LLMs

Paper presented at the DAC 2026 Work in Progress (WIP) Poster Session.

This work is licensed under a Creative Commons Attribution 4.0 International License.

struggle to interpret unless these artifacts are transformed into text or handled by specialized models. Because natural language is inherently ambiguous, converting specs into a formal representation is critical to avoid misinterpretation. Meanwhile, RTL provides full visibility into the design's microarchitecture, enabling LLMs to generate internal assertions that remain reusable as long as those parts of the design remain stable. This snapshot methodology is particularly valuable for AI-driven design, where rapid changes are difficult to manually audit. SVApshot's golden reference toolflow is orthogonal to prior works and could even adopt different assertion generation rules. The current version of SVApshot uses the rules from AutoSVA2 [21]. SVApshot's generation based on reference RTL implementations is ***also applicable to automatically creating formal testbenches for benchmarking of LLMs at hardware design***, which is more exhaustive than simple functional equivalence comparison and provides more verification visibility than LEC (Logical Equivalence Checking), i.e. offers a direct view of the mismatching logic, with a concrete failing assertion, rather than a general output mismatch that requires debugging. Several benchmarks have emerged to evaluate LLMs for RTL generation using coarser-grained metrics like functional equivalence or simulation-based testbenches. These approaches would benefit from using SVApshot to curate assertion suites as *property checklists* for exhaustively evaluating LLM-generated RTL. RTLLM [14] provides an open benchmark suite comprising design descriptions, test cases, and golden RTL implementations to assess LLM capabilities in RTL generation. VerilogEval [15] focuses on specific tasks like code completion and spec-to-RTL translation. TuRTLe [7] offers a unified evaluation framework, integrating multiple existing benchmarks (including VerilogEval and RTLLM) to assess LLM performance across a spectrum of RTL generation tasks. SVApshot is uniquely positioned to facilitate the benchmarking of LLM-generated RTL. Instead of relying solely on simulation-based testbenches for benchmarking, which may miss corner-case bugs, prior work can leverage SVApshot to create comprehensive, high-quality, and formally verifiable assertion suites. SVApshot introduces several **key innovations** that enable this snapshot methodology:

- ① Implements an automated iterative repair loop that attempts to fix failing assertions using LLM-guided debugging, addressing a critical gap in existing tools that typically treat generated assertions as final outputs.
- ② Provides sophisticated duplicate detection and assertion set extension capabilities that ensure comprehensive coverage while avoiding redundancy through conjunctive normal form (CNF) analysis.
- ③ Integrates these features through an automatic and modular pipeline that begins with automated formal testbench generation, proceeds through LLM-based assertion generation with systematic syntax correction, expands assertion sets through targeted prompting, and concludes with iterative semantic repair using formal verification feedback.

This end-to-end LLM-assisted automation, combined with robust error handling and systematic refinement, enables SVApshot to generate testbenches that serve both as verification artifacts and as benchmarking instruments for evaluating AI-generated RTL. Figure 1 illustrates the complete toolflow through a detailed diagram, which Algorithm 1 further details with pseudocode.

Algorithm 1 SVApshot Toolflow

Require: RTL module M
1: **Stage 1:** Generate formal testbench environment for M
2: **Stage 2:** $A_{initial} \leftarrow$ LLM.generateAssertions(M, rules)
3: **Stage 3:** $A_{corrected} \leftarrow$ correctSyntax($A_{initial}$)
4: **Stage 4:** Initialize property database with $A_{corrected}$
5: **repeat**
6: $A_{pot_unique} \leftarrow$ removeDupl(A_{new}, prop_db)
7: $A_{pot_unique_corr} \leftarrow$ correctSyntax(A_{pot_unique})
8: $A_{unique} \leftarrow$ removeDupl($A_{pot_unique_corr}$, prop_db)
9: $A_{corrected} \leftarrow A_{corrected} \cup A_{unique}$
10: Update property database (prop_db)
11: **until** $A_{unique} = \emptyset$ ▷ All new assertions are duplicates
12: **Stage 5:** $results \leftarrow$ formalVerify($A_{corrected}$, M)
13: $A_{proven} \leftarrow \emptyset$, $A_{cex} \leftarrow \emptyset$
14: **for** each assertion $a \in A_{corrected}$ **do**
15: **if** $results[a]$ = PROVEN **then**
16: $A_{proven} \leftarrow A_{proven} \cup \{a\}$
17: **else if** $results[a]$ = CEX **then**
18: $repaired \leftarrow$ false, $attempts \leftarrow 0$
19: **while** $\neg repaired$ **and** $attempts < 5$ **do**
20: $a_{fixed} \leftarrow$ LLM.fixSVA(a, $results[a].cex$, M)
21: $result \leftarrow$ formalVerify(a_{fixed}, M)
22: **if** $result$ = PROVEN **then**
23: $A_{proven} \leftarrow A_{proven} \cup \{a_{fixed}\}$
24: $repaired \leftarrow$ true
25: **end if**
26: $attempts \leftarrow attempts + 1$
27: **end while**
28: **if** $\neg repaired$ **then**
29: $A_{cex} \leftarrow A_{cex} \cup \{a\}$
30: **end if**
31: **end if**
32: **end for**
33: **return** snapshot(A_{proven}, A_{cex})

2 The SVApshot Toolflow

SVApshot is a modular framework that integrates LLM-based assertion generation with formal verification techniques. It removes humans from the generation loop by treating the input RTL module as a golden reference. By establishing trust in the RTL, we assume all proof results are correct, eliminating the notion of false positives (assertions that are passing but the RTL is wrong). This assumption constitutes the cornerstone of SVApshot, which enables the semantic repair capability (Stage 5 of Algorithm 1). Section 2.1 details each toolflow stage, and Sections 2.2 and 2.3 present the use cases for which SVApshot's methodology introduces a novel solution.

2.1 Toolflow Overview

The SVApshot toolflow comprises five sequential stages:

(1) **Testbench Generation:** Automated scripts generate the necessary formal testbench environment for the target RTL module. This includes directory structure, assertion files, binding files, and compilation scripts required for the Formal Property Verification (FPV) tool.
(2) **Initial Assertion Generation:** The LLM generates an initial assertion set using the RTL module and curated SVA generation rules as input. The prompt combines the golden RTL with established SVA writing guidelines to enhance semantic relevance.
(3) **Syntax Correction:** The framework corrects syntactic errors in the initial assertion set through a three-phase approach. Automated methods first address common errors: prefixing internal signals, enum values, parameters, and localparams with module names (**3.A** in Figure 1).
The system then resolves assertion name collisions (**3.B** in Figure 1). These seldom happen, but we have observed instances of initial assertion sets with duplicated assertion names. Following compilation attempts with the FPV tool, remaining syntax errors trigger an LLM-based correction process that prompts the LLM with the RTL code, current assertions, and specific error messages (**3.C** in Figure 1). This iterative process continues until compilation succeeds. In the rare case that assertions prove very difficult to fix, a protection mechanism stops the LLM-based syntax correction after 10 attempts. When this limit is reached, the tool progressively removes assertions from the set until a subset of syntactically correct assertions remains.
(4) **Assertion Set extension:** Syntactically correct assertions are parsed and stored in a property database in conjunctive normal form (CNF) for reliable duplicate detection; CNF processing prevents wrongly considering assertions as different when they differ only in signal ordering (e.g., $a \wedge b$ and $b \wedge a$), or are logically equivalent (e.g., $a \implies b$ and $\neg a \vee b$). The delay of every concurrent assertion is also stored for comparison. We remove the delay from the expression so that the parser only processes the implication, AND, OR, and NOT operands to do the CNF conversion. We define assertions as *duplicated* when they are logically equivalent and have the same delay. If two assertions are different but have the same name, the tool adjusts the names automatically to prevent syntax errors. The LLM generates additional assertions using the RTL and existing properties as context. Automated duplicate detection precedes syntax correction to optimize computational efficiency, i.e., we avoid running the FPV tool on duplicate assertions. New assertions undergo syntax correction via Stage 3, followed by a final duplicate detection step. This assertion set extension process continues until the LLM produces only duplicate assertions. Thus, our de-duplication step serves as both a computationally-efficient filter and a reliable termination condition for the set extension process.
(5) **Iterative Semantic Repair:** The semantic correction loop addresses failing assertions through LLM-based repair. Each iteration prompts the LLM with RTL code, one failing assertion, and previously attempted fixes. The framework limits repair attempts to 5 iterations per assertion to ensure termination. The toolflow concludes after attempting repairs for all failing assertions.

2.2 Use case: Snapshot-Based Verification of AI-Driven Design

The primary use case for SVApshot is verifying the preservation of functionality in RTL designs undergoing AI-assisted modifications. This application leverages formal assertions as regression tests to detect unintended behavioral changes.

- **Establishing a Baseline:** SVApshot generates a comprehensive set of SVA properties for an existing RTL module through the five-stage toolflow. After formal verification and iterative repair, these properties constitute an initial behavioral snapshot. This snapshot serves as a trusted baseline for subsequent design modifications.
- **Verifying AI-Generated Changes:** Modified RTL undergoes verification against the established snapshot to detect behavioral regressions. End-to-end assertions involving only inputs and outputs remain applicable when the module interface is preserved. Internal assertions may require selective reuse based on the specific signals and implementation details affected by modifications. The key principle of SVApshot is that failing assertions identify potential issues introduced by changes, providing automated regression detection.
- **Incremental Snapshots:** The snapshot methodology supports incremental updates as designs evolve and new functionalities undergo validation. Updated snapshots reflect the current trusted design state while maintaining historical verification artifacts.

This methodology enhances confidence in AI-assisted RTL development by providing automated formal verification of functional preservation across design iterations. This approach aligns well with test-driven development (TDD) [4].

2.3 Use case: Benchmarking LLMs for RTL Generation

SVApshot enables fine-grained benchmarking of LLM capabilities in hardware design tasks beyond simple functional equivalence. Functional equivalence (determined by tools like logic equivalence checkers [10]) between reference and candidate designs provides only binary pass/fail metrics, which are insufficient to assess the quality and nuanced aspects of LLM output. Although comprehensive benchmarking of LLM-generated RTL is beyond the scope of this paper, it provides a framework to enable such evaluation.
Assertion-Based Scoring: LLM-generated RTL can undergo evaluation against reference modules' snapshots to assess design quality. The number of passing properties directly serves as a quality metric for RTL generation. More sophisticated metrics could include weighted scoring based on assertion complexity (number of CNF clauses, number of signals, etc.).

3 Experimental Methodology

Module Selection: Table 1 lists the 14 RTL modules evaluated and provides a brief description for each of them. We evaluate SVApshot's capabilities using a diverse set of RTL modules spanning different complexity levels and design domains. The module selection includes components from both internal company projects and open-source components from the Sargantana processor core [3], chosen to represent realistic hardware design scenarios.
Experimental Configuration: To sense what the impact of LLM sizes is on the SVApshot toolflow, we used the models in Table 2. The temperature value was chosen to balance assertion diversity with generation consistency and reproducibility, while the max token count was set based on the observation that each LLM task required around 4000 tokens. We ran the models using the NVIDIA NIM API [18]. For formal property verification and coverage analysis, we utilized Cadence JasperGold v2021.12 [5]. The repository `dac-2026-artifacts` contains the final assertion testbenches for `div_unit`, `div_unit_4bits`, `mul_unit`, `ptw`, `ptw_arb`, and `pseudoLRU`, and the script that re-proves one experiment or the full set with JasperGold and with VC Formal.

Table 1: RTL Modules Used for Evaluation.

Module	Description
popcount	Parameterizable pop-count/reduction
skipper_counter	Configurable state counter
edge_detector	Signal edge detector
alu (Arithmetic Logic Unit)	Asynchronous ALU
dynamic_target_event_counter	Parallel event counter
score_board_scalar	Scalar datapath scoreboard
branch_unit	Branch execution control unit
tlb (Transaction Lookaside Buffer)	Memory Management Unit TLB
ptw (Page Table Walker)	Memory Management Unit PTW
ptw_arb	PTW request arbiter
pseudoLRU	Pseudo-LRU controller
div_unit	Pipelined 32/64-bit division unit
div_unit_4bits	4-bit division unit
mul_unit	Pipelined 32/64-bit multiplication unit

Table 2: LLMs used for experimentation.

Model	Modules Evaluated	Configuration
GPT-4.1-mini [19]	14	temp = 0.2, max_token = 8192
Llama3.1 405B[17]	6	temp = 0.2, max_token = 8192
Llama3.3 70B [17]	6	temp = 0.2, max_token = 8192

Evaluation Metrics: Given that formal verification tools have been around for decades and have proved their usefulness at catching bugs [22], it would not add much value to the reader for us to prove that a formal testbench with full coverage would catch any bug that a code refactor may introduce. Thus, we evaluate SVApshot effectiveness using checker coverage as the primary metric. Checker coverage quantifies the portion of the design state space explored by the assertion set, providing a reliable indicator of assertion quality and completeness. Higher checker coverage indicates more comprehensive verification of the RTL module's behavioral space. Secondary metrics include initial assertion pass rates and semantic correction success rates. These and other metrics enable analysis of both coverage progression and assertion set evolution throughout the flow, as well as assessment of the quality of generated assertions and the effectiveness of the repair process. The reported checker coverage results are calculated using only the proven assertions to establish the coverage baseline achieved by the snapshot. Any remaining failing assertions, if corrected, would potentially contribute additional coverage to the design, especially since our methodology already detects and removes equivalent assertions. ***Measuring the coverage of the generated SVA evaluates the goodness of SVApshot at both use cases*** defined in Sections 2.2 and 2.3, since the more thorough the formal testbench it creates, the more exhaustive is the snapshot at capturing future RTL deviations and the testbench at benchmarking nuanced RTL-generation capabilities.

4 Experimental Results

Section 4.1 presents comprehensive experimental results of applying SVApshot to a diverse set of RTL modules, ranging from 100 to 400 lines of code (LOC), including complex hierarchical designs with submodules. Section 4.2 presents the CEX correction analysis. Section 4.3 presents an analysis of the runtime and cost of SVApshot, and lastly, a validation example is presented in Section 4.4. Table 3 results demonstrate SVApshot's effectiveness across different module complexities and sources. For the `tlb`, `ptw` and `div_unit` modules, we generated assertions for their submodules as well, given their significantly larger size and complexity, providing a more comprehensive evaluation of SVApshot's capabilities on hierarchical designs. Table 3 shows results for these modules with assertions for submodules, whereas for other modules without submodules (e.g., `skipper_counter` and `dynamic_target_event_counter`) we include only assertions for the top-level module. Thus, further coverage could be achieved for these top-level modules by adding assertions for submodules, since the coverage tool reports coverage for the entire hierarchy.

4.1 Snapshot Coverage Across Modules

The experimental results demonstrate several key findings across multiple dimensions of analysis. **Effect of LLM choice:** The first section of Table 3 shows modules evaluated only with GPT 4.1 mini, while the latter three sections show modules evaluated with several LLMs (and bold the highest coverage for each module). We notice that while GPT 4.1 mini performs well overall, for some modules, the Llama models surpass the achieved coverage. The 405-billion parameter Llama version is better than or equal to the 70-billion one, and results are vastly better for `mul_unit`. We anticipate that future, more capable AI models (especially those fine-tuned on SystemVerilog) would yield even higher coverage, with the same tooflow. **Coverage Improvement Analysis:** SVApshot achieved significant coverage improvement from initial SVA generation (Stage 2 in Figure 1) to the end of set extension (Stage 4) across the majority of evaluated modules. The `alu` module achieves 100% initial coverage, an exceptional result, due to its simple functional specifications, which are easily translated into SVA properties. The most substantial improvement occurred in the `popcount` module, where coverage increased from up to 93.9%. This module only has

Table 3: Experiment results. First section groups RTL modules evaluated with GPT4.1-mini; the rest cover the ones evaluated with 3 LLMs. Topology is either Sequential (S) or Combinational (C). Number of unique submodules (Sub). Initial (Init) results represent the results obtained after finishing Stage 3.

RTL Module	LLM	Topology	Source	LOC	Sub (#)	Init Assrt (#)	Init Cov (%)	Pass Before Stage 5 (#/Total)	Final Pass (#/Total)	Final Cov (%)
popcount	GPT 4.1 mini	S	Internal	139	2	5	0	0/59	3/59	93.9
skipper_counter	GPT 4.1 mini	S	Internal	165	2	22	34.1	4/22	4/22	53.5
edge_detector	GPT 4.1 mini	S	Internal	24	0	5	73.3	19/19	19/19	73.3
alu	GPT 4.1 mini	C	Internal	173	0	28	100	18/46	35/46	100
dynamic_target_event_counter	GPT 4.1 mini	S	Internal	230	1	14	62.8	6/14	6/14	62.8
score_board_scalar	GPT 4.1 mini	S	Sargantana Core	96	0	22	55.7	11/22	12/22	86.0
branch_unit	GPT 4.1 mini	C	Sargantana Core	140	0	14	14.4	1/14	12/14	42.0
tlb (with submodules)	GPT 4.1 mini	S	Sargantana Core	372	1	8	23.7	33/43	37/43	39.3
pseudoLRU	GPT 4.1 mini	S	Sargantana Core	137	0	8	68.7	22/35	29/35	70.7
ptw (with submodules)	**GPT 4.1 mini**	S	Sargantana Core	419	2	40	47.9	75/107	96/107	**52.8**
ptw_arb	**GPT 4.1 mini**	S	Sargantana Core	115	0	17	95.5	28/37	33/37	**95.7**
pseudoLRU	GPT 4.1 mini	S	Sargantana Core	137	0	8	68.7	22/35	29/35	70.7
ptw (with submodules)	Llama 3.3 70b	S	Sargantana Core	419	2	54	43.3	137/221	162/221	44.6
ptw_arb	Llama 3.3 70b	S	Sargantana Core	115	0	28	88.7	24/49	32/49	89.0
pseudoLRU	**Llama 3.3 70b**	S	Sargantana Core	137	0	12	71.7	87/106	95/106	**71.7**
ptw (with submodules)	Llama 3.1 405b	S	Sargantana Core	419	2	5	28.8	68/116	86/116	43.4
ptw_arb	Llama 3.1 405b	S	Sargantana Core	115	0	8	69.4	14/29	19/29	88.7
pseudoLRU	**Llama 3.1 405b**	S	Sargantana Core	137	0	11	68.7	21/45	32/45	**71.7**
div_unit (with submodules)	GPT 4.1 mini	S	Internal	241	1	17	84.2	74/128	96/128	85.6
div_unit_4bits	GPT 4.1 mini	C	Internal	100	0	9	91.5	3/10	8/10	96.6
div_unit (with submodules)	Llama 3.3 70b	S	Internal	241	1	60	76.0	129/170	132/170	83.9
div_unit_4bits	**Llama 3.3 70b**	C	Internal	100	0	17	99.7	34/37	35/37	**99.7**
div_unit (with submodules)	**Llama 3.1 405b**	S	Internal	241	1	36	74.5	31/65	51/65	**89.5**
div_unit_4bits	**Llama 3.1 405b**	C	Internal	100	0	16	91.6	20/23	21/23	**99.7**
mul_unit	**GPT 4.1 mini**	S	Internal	294	0	40	92.6	51/115	81/115	**98.9**
mul_unit	Llama 3.3 70b	S	Internal	294	0	29	0.7	4/45	10/45	1.76
mul_unit	Llama 3.1 405b	S	Internal	294	0	10	1.5	24/67	37/67	70.3

one output signal, and the three correct assertions targeted it, thus increasing the coverage significantly. Modules that exhibited modest coverage increases typically experienced limited assertion set extension during the extension phase, rather than extension without corresponding coverage gains, as occurred, for example, in the `div_unit_4bits` module. The `edge_detector` module maintained its initial 73.3% coverage, as the generated assertions were already comprehensive and no additional meaningful assertions were discovered during extension. **RTL Topology Impact:** The results reveal distinct patterns based on RTL topology and functional domain. Units `alu` and `div_unit` exhibited high initial coverage and strong assertion correction rates, though not necessarily the highest coverage increases due to already comprehensive initial coverage. Control modules like `ptw` demonstrated greater resistance to semantic correction, likely due to their complex behavioral specifications. **Assertion Generation Quality:** The initial pass rates of generated assertions were generally high across most modules, indicating that LLMs can produce syntactically and semantically reasonable assertions with minimal repair. Modules with lower initial pass rates typically involve complex control logic or timing-sensitive behaviors that are inherently more difficult to specify correctly. The semantic correction phase successfully improved assertion pass rates in most cases, with notable exceptions in modules requiring domain-specific assumptions about input constraints. **Scalability Considerations:** The analysis of the `tlb` and `ptw` modules reveals the challenges associated with scaling to larger module sizes. SVApshot addresses entire modules without partitioning, thus demonstrating its ability to achieve meaningful coverage even on complex designs when applied holistically. Moreover, since traditional FPV approaches for modules of this size may already employ divide-and-conquer strategies due to state space explosion concerns, further modularization of the design could enable SVApshot to achieve better results as observed with smaller modules.

4.2 Counterexample (CEX) Correction

Converting CEXs into passing assertions boosts the coverage of the SVA testbench created by SVApshot. Our methodology performs several LLM-driven attempts to fix failing (CEX) assertions (Stage 5 in Algorithm 1). However, this process may not address all assertion CEXs, since assertions may have semantic errors, or assumptions could be missing for semantically correct assertions. Thus, after using SVApshot, an engineer could carefully review whether assumptions are missing for module inputs that would lead to spurious CEXs, as well as revise failing assertions for semantic errors. Next, we analyze whether there are trivial assumption-fixable CEXs and the effectiveness of LLM-driven corrections. **Assumptions can fix CEXs and increase coverage:** The FPV tool revealed that some assertions are semantically correct but fail (i.e., lead to CEX) due to underconstrained inputs. In the scoreboard module, the failing assertion `as__OcupDivUnitQ1ShiftRightNoSet` was corrected by adding an assumption constraining the flush signal to zero, confirming the hypothesis that input assumptions can resolve some CEXs reported by the FPV tool. This finding suggests that SVApshot's current limitation in generating input assumptions leads to false negative results that could be addressed through assumption synthesis. However, as the literature [22] tells us, prematurely adding

assumptions may lead to false positives, so we decided not to automatically add assumptions in our methodology and leave that to the engineer.**Semantic Correction Attempt Analysis:** After extracting execution data for the final phase of the flow, we measured the effectiveness of attempting to correct failing assertions (CEX). The majority of the assertions that the LLM corrected (went from CEX to Proof) were done so on the first attempt. For modules evaluated on all LLMs, GPT 4.1 mini corrected 27% of the assertions on the first attempt, and the total percentage of corrected assertions in 5 attempts was 43.3%; for Llama 3.3 70b 17.3% and 25.8%; and for Llama 3.1 405b 15.6% and 43.8%. We could potentially reduce the attempt count significantly from five attempts (and expect most of the LLM-fixable properties to be corrected) to reduce the LLM cost (albeit low as shown in Section 4.3), LLM latency, and FPV runtime, since this phase takes the longest, and is entirely dependent on the LLM and the FPV tool.

4.3 SVApshot Runtime and Cost

Dissecting the time that each of the stages in the SVApshot toolflow takes (detailed in Algorithm 1), we found that generating the initial testbench (Stage 1 and 2) takes the order of a few seconds, being the most residual execution time. Initial syntax correction of the initial assertion set (Stage 3) required on average 2 minutes (3.3% of the total time) The set extension (Stage 4) consumed 9 minutes on average (19.3% of the total). Lastly, Semantic correction (Stage 5) with 5 attempts required 55 minutes (77.4% of the total), due to the LLM call latency and FPV tool runtime. This is why the reduction of the attempt limit we give the LLM in Stage 5 would be a way to reduce the execution time and obtain faster snapshots. Further dissection of Stages 3-5 showed that the FPV tool usage represented 69.4% of the total execution time on average. Despite these high execution times, we evaluated the cost of each experiment based on the input and output tokens used and prices from inference providers for each of the models, and obtained that the average cost for a snapshot was 0.5€, which again could be reduced if the attempt limit in Stage 5 were tuned down.

4.4 Validating The Snapshot Methodology

Validation Through Bug Injection: To evaluate the practical utility of generated assertions, we manually introduced a representative bug in the scoreboard module. The bug involved incorrectly setting `inst_d[2]` instead of `inst_d[1]` when a 64-bit multiply instruction is issued, creating an off-by-one timing error. The assertion `as__SetMul64SetsInstD1` (Listing 1) successfully detected this bug, demonstrating that SVApshot generates assertions capable of capturing meaningful design intent and detecting realistic implementation errors.

```
as__SetMul64SetsInstD1: assert property((set_mul_64_i) |-> (
      scoreboard_sargantana.inst_d[1] == 1'b1));
```

Listing 1: Snapshot assertion for the scoreboard module.

Validation Through LLM-based Refactoring: We performed an experiment of LLM-based refactor of the `ptw_arb` module using Cursor. We asked the code agent (claude-4-sonnet) to refactor the original `ptw_arb` module, keeping the module interface signals intact. After refactoring its finite-state machine (FSM) and running the snapshot from the original module through the FPV tool, we saw a CEX for an assertion that passed originally (Listing 2).

```
as__IdleStateOnlyDtlbReqOutput:assert property(
    (ptw_arb.current_state_q==ptw_arb.IDLE &&
     dtlb_ptw_i.req.valid && !itlb_ptw_i.req.valid &&
     !ptw_arb.itlb_req_pending_q) |->
    (tlb_ptw_o==dtlb_ptw_i));
```

Listing 2: Failing snapshot assertion after LLM refactor of RTL.

This assertion captures the behavior that when the PTW arbiter is in `IDLE` state and only the DTLB has a valid request (no ITLB request), the output should directly connect the DTLB communication interface. This behavior breaks in the version edited by AI since the refactored version splits the `next_state` signals from the output signals into two blocks and then confuses the state for which `tlb_ptw_comm_o` was supposed to change, from `IDLE` to `SERVING_DTLB`. This example demonstrates the value of having the snapshot before refactoring or modifications, as well as the ability for the snapshot to detect bugs in the refactored version.

5 Discussion and Conclusion

This paper introduces SVApshot, a systematic framework for automated SVA generation that addresses critical verification challenges in AI-driven RTL design workflows. The increasing reliance on LLMs for RTL code generation and modification necessitates robust mechanisms for ensuring functional correctness and regression testing. Although SVA generation has been explored previously, our snapshot-based approach enables a systematic refinement of assertions to conform to an RTL implementation. SVApshot provides a framework for capturing design functionality of an RTL module as formal assertions, enabling two use cases: (1) establishing an implementation baseline for regression testing of subsequent modifications, and (2) generating testbenches for modules used to benchmark LLM-aided RTL design. Our methodology accomplishes this by implementing an iterative repair approach that systematically addresses both syntactic and semantic errors in LLM-generated assertions through automated correction loops, significantly improving assertion quality over single-pass generation methods. Overall, SVApshot's toolflow combines assertion generation, duplicate detection, assertion set extension, and formal verification in a cohesive framework, reducing manual intervention. Our experimental evaluation across diverse RTL modules demonstrates the framework's effectiveness in achieving high coverage across a set of 14 diverse, complex modules with up to 400 lines of SystemVerilog. The semantic correction phase successfully improved assertion pass rates in most evaluated designs, while the assertion set extension mechanism generated comprehensive property sets without logical redundancy. The validation through deliberate bug injection confirms that SVApshot generates assertions capable of detecting realistic implementation errors, both human- and LLM-generated. Moreover, the experiments reveal interesting aspects of LLM-based assertion generation. Arithmetic and datapath modules consistently achieved higher initial coverage and correction rates compared to complex control logic implementations. When comparing the testbenches generated by various LLMs, we observed higher coverage with newer (GPT 4.1 mini) or larger (Llama 405b) LLMs, which suggests that future improvements in LLM capabilities could further enhance

coverage results of the SVApshot methodology. However, we acknowledge some **limitations** of our methodology: (1) SVApshot does not verify the correctness of the input RTL, but rather aims to verify that future modifications do not break the current functionality. (2) The semantic repair process, while effective in most cases, does not address CEXs requiring domain-specific input assumptions. Future research directions include extending the framework to generate adequate assumptions on RTL module inputs (to avoid spurious CEXs), as well as developing more sophisticated metrics for assertion relevance when using assertions as weighted scores for LLM benchmarking. The application of SVApshot to comprehensive LLM benchmarking could be a paper on its own, to standardize evaluation methodologies for AI-generated RTL designs. While challenges remain in handling complex control logic and assumption generation, the demonstrated effectiveness and inexpensive execution across a diverse set of RTL modules suggest that this work represents a significant step toward trustworthy AI-assisted hardware verification, providing both the theoretical framework and practical tools necessary to maintain design integrity in an era of increasingly automated hardware development.

References

[1] Mohammad Akyash, Kimia Azar, and Hadi Kamali. 2025. RTL++: Graph-enhanced LLM for RTL Code Generation. *arXiv preprint arXiv:2505.13479* (2025).

[2] Yunsheng Bai, Ghaith Bany Hamad, Syed Suhaib, and Haoxing Ren. 2025. AssertionForge: Enhancing Formal Verification Assertion Generation with Structured Representation of Specifications and RTL. *arXiv preprint arXiv:2503.19174* (2025).

[3] Barcelona Supercomputing Center. 2025. Sargantana. https://github.com/bsc-loca/sargantana.

[4] Jon Buckingham. 2016. Formal for Designers. In *Agile Test Driven Dev. for ASIC*. https://doi.org/wp-content/uploads/2016/VerificationFutures/9_HPE_JohnBuckingham_(ASICPresentation).pdf

[5] Cadence Design Systems Inc. 2015. JasperGold Apps User's Guide.

[6] Alessandro Danese, Nicol'o Dalla Riva, and Graziano Pravadelli. 2017. A-TEAM: Automatic template-based assertion miner. In *Proc. DAC*. 37:1–37:6. https://doi.org/10.1145/3061639.3062206

[7] Dario Garcia-Gasulla, Gokcen Kestor, Emanuele Parisi, Miquel Albertí-Binimelis, Cristian Gutierrez, Razine Moundir Ghorab, Orlando Montenegro, Bernat Homs, and Miquel Moretó. 2025. TuRTLe: A Unified Evaluation of LLMs for RTL Generation. *arXiv preprint arXiv:2504.01986v1* (Mar 2025). arXiv:2504.01986 [cs.AR] https://arxiv.org/abs/2504.01986

[8] Samuele Germiniani and Graziano Pravadelli. 2022. HARM: A Hint-Based Assertion Miner. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4277–4288.

[9] Bo-Yuan Huang, Hongce Zhang, Pramod Subramanyan, Yakir Vizel, Aarti Gupta, and Sharad Malik. 2018. Instruction-Level Abstraction (ILA): A Uniform Specification for System-on-Chip Verification. *ACM Transactions on Design Automation of Electronic Systems (TODAES)* 24, 1 (2018), 1–24.

[10] Shi-Yu Huang and Kwang-Ting Tim Cheng. 2012. *Formal equivalence checking and design debugging*. Vol. 12. Springer Science & Business Media.

[11] Mubashir ul Islam, Humza Sami, Pierre-Emmanuel Gaillardon, and Valerio Tenace. 2024. AIvril: AI-Driven RTL Generation with Verification In-The-Loop. *arXiv preprint arXiv:2409.11411* (2024).

[12] Wenchao Li, Lee Dworkin, and Sanjit A. Seshia. 2013. Mining hardware assertions with guidance from static analysis. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 32, 6 (2013), 952–965.

[13] Shang Liu, Wenji Fang, Yao Lu, Jing Wang, Qijun Zhang, Hongce Zhang, and Zhiyao Xie. 2024. RTLCoder: Fully Open-Source and Efficient LLM-Assisted RTL Code Generation. *arXiv preprint arXiv:2312.08617* (2024).

[14] Shang Liu, Yao Lu, Wenji Fang, Mengming Li, and Zhiyao Xie. 2024. OpenLLM-RTL: Open Dataset and Benchmark for LLM-Aided Design RTL Generation and Verification. In *Proceedings of the ACM/IEEE International Conference on Computer-Aided Design (ICCAD '24)*. https://arxiv.org/abs/2503.15112 Preprint arXiv:2503.15112.

[15] Shang Liu, Zhiyuan Yan, Wenji Fang, Yao Lu, Mengming Li, Hongce Zhang, and Zhiyao Xie. 2023. VerilogEval: A Large-Scale Dataset for Verilog Code Generation and Understanding. *arXiv preprint arXiv:2310.09231* (Oct 2023). arXiv:2310.09231 [cs.SE] https://arxiv.org/abs/2310.09231

[16] Anand Menon, Debjit Pal, Deeksha Nandal, Vaishnavi Pulavarthi, Soham Dan, Shamik Kundu, and Amisha Srivastava. 2025. Spec2Assertion: Generating SystemVerilog Assertions from Design Specifications using Large Language Models with Chain-of-Thought Prompting and Progressive Regularization. *arXiv preprint arXiv:2505.07995v2* (May 2025). arXiv:2505.07995 [cs.AI] https://arxiv.org/abs/2505.07995

[17] Meta. 2025. Llama 3.1. https://ai.meta.com/blog/meta-llama-3-1/.

[18] NVIDIA. 2025. NVIDIA NIM. https://build.nvidia.com/explore/discover.

[19] OpenAI. 2025. GPT-4.1-mini. https://platform.openai.com/docs/models/gpt-4.1-mini.

[20] Marcelo Orenes-Vera, Aninda Manocha, David Wentzlaff, and Margaret Martonosi. 2021. AutoSVA: Democratizing Formal Verification of RTL Module Interactions. In *ACM/IEEE Design Automation Conference (DAC)*. 535–540. https://doi.org/10.1109/DAC18074.2021.9586118

[21] Marcelo Orenes-Vera, Margaret Martonosi, and David Wentzlaff. 2023. Using LLMs to Facilitate Formal Verification of RTL. arXiv:2309.09437 [cs.AR] https://arxiv.org/abs/2309.09437

[22] Erik Seligman, Tom Schubert, and MV Achutha Kiran Kumar. 2015. *Formal verification: an essential toolkit for modern VLSI design*. Morgan Kaufmann.

[23] Zhiyuan Yan, Wenji Fang, Mengming Li, Min Li, Shang Liu, Zhiyao Xie, and Hongce Zhang. 2025. AssertLLM: Generating Hardware Verification Assertions from Design Specifications via Multi-LLMs. In *30th Asia and South Pacific Design Automation Conference (ASPDAC '25)*.